



Low Level Design

Global Mobility Application Analyzer

Written By	Chandrajyothi Biju, Fidha Manaph
Document Version	2
Last Revised Date	2 - June-2026

Contents

1. Introduction.....	1
1.1. What is a Low-Level design document?.....	1
1.2. Scope.....	1
2. Architecture.....	2
3. Architecture Description.....	3
3.1. Data Description.....	3
3.2. Data Ingestion.....	3
3.3. Exploratory Data Analysis.....	3
3.4. Data Validation.....	3
3.5. Drift Detection.....	3
3.6. Data Preprocessing.....	4
3.7. Feature Engineering.....	4
3.8. Data Transformation.....	4
3.9. Model Training and Hyperparameter Tuning.....	4
3.10. Model Evaluation.....	4
3.11. Model Pusher.....	4
3.12. AWS Cloud Setup.....	6
3.13. FastAPI Deployment.....	6
3.14. Prediction Pipeline.....	6
3.15. Approval Prediction.....	6
4. Unit Test Cases.....	7

1. Introduction

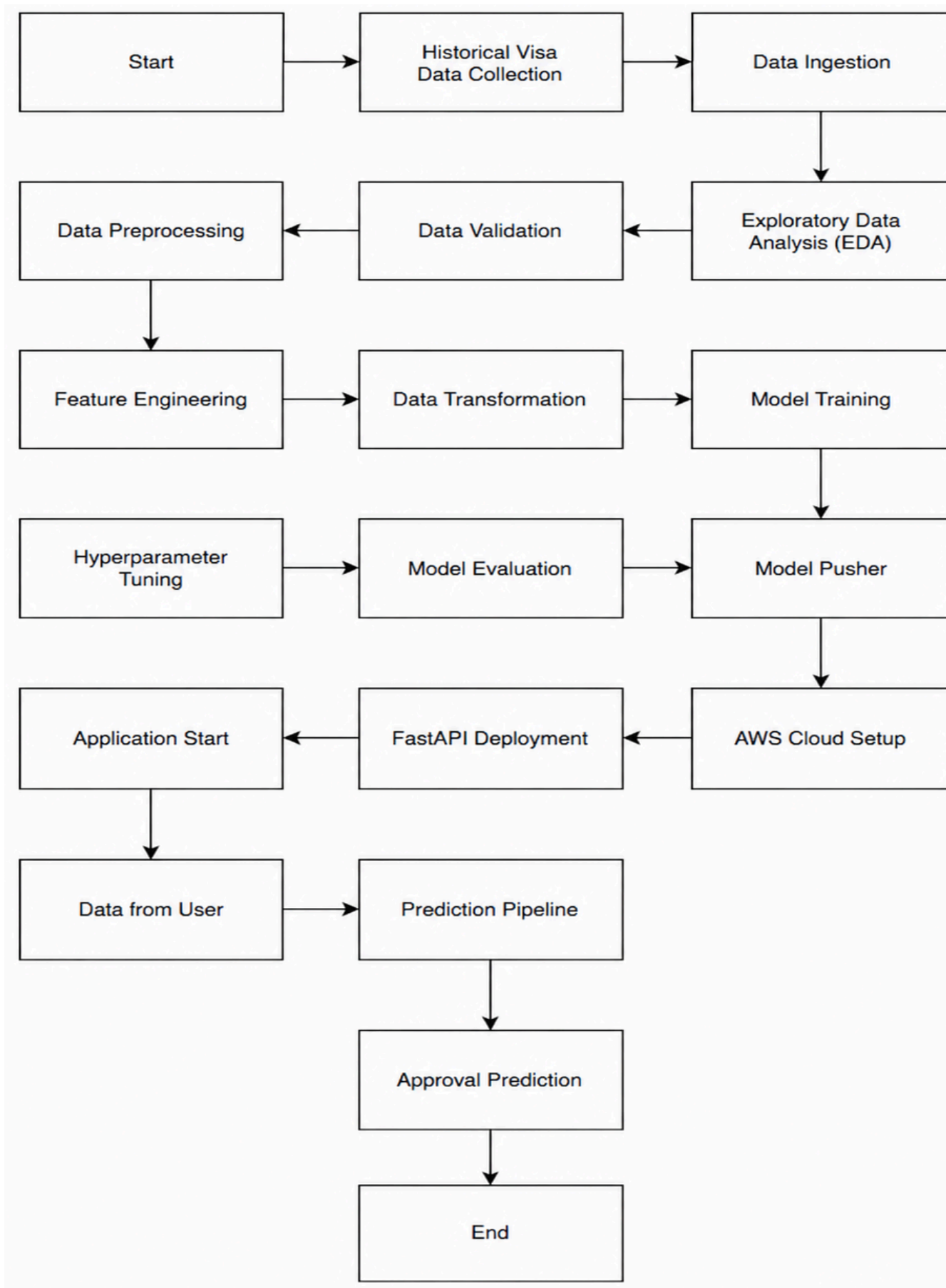
1.1. What is a Low-Level design document?

The goal of the Low-Level Design (LLD) document is to provide the internal logical design of the actual program code for the Visa Approval Prediction System. The LLD describes the detailed structure of the system, including class diagrams, methods, database interactions, and relationships between components. It explains how the machine learning pipeline, data preprocessing, model training, prediction, and user interface modules interact with each other so that developers can directly implement the system from the document.

1.2. Scope

Low-Level Design (LLD) is a component-level design process used to define the software architecture, data structures, source code organization, and machine learning workflow of the Visa Approval Prediction System. The system predicts visa approval or denial using historical immigration data and helps consultants, employers, and applicants assess approval chances, identify risk factors, and automate initial case screening.

2. Architecture



3. Architecture Description

3.1. Data Description

The dataset used for this project is the Easy Visa Dataset, containing approximately 25,480 visa application records. The dataset consists of applicant and employer-related features such as continent, education level, work experience, job training requirement, prevailing wage, number of employees, unit of wage, and year of establishment. The target variable, **case_status**, indicates whether a visa application was **Certified** or **Denied**.

3.2. Data Ingestion

Data ingestion is the first stage of the pipeline, where visa application records are extracted from the MongoDB database and converted into structured Pandas DataFrames. The dataset is stored in a feature store and automatically split into training and testing datasets for model development and evaluation.

3.3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand feature distributions, identify missing values, detect class imbalance, analyze correlations, and discover patterns influencing visa approval decisions. Both univariate and multivariate analyses were conducted to understand the relationship between applicant and employer attributes and visa status.

3.4. Data Validation

Data validation ensures that the dataset structure is correct before model training. Validation checks whether all required columns are present, verifies numerical and categorical features, and confirms that the dataset matches the predefined schema.yaml configuration. This prevents invalid or inconsistent data from entering the pipeline.

3.5. Drift Detection

Data drift detection identifies changes in incoming data compared to the data used during model training. In this project, Evidently AI was used to compare feature distributions between training and testing datasets. Detecting drift helps maintain model reliability and prevents performance degradation over time.

3.6. Data Preprocessing

Data preprocessing includes handling missing values, removing duplicate records, encoding categorical variables, and preparing the dataset for machine learning. Mean imputation was used for numerical features, while mode imputation was applied to categorical features. These steps ensure data quality and consistency before transformation.

3.7. Feature Engineering

Feature engineering was performed to improve model performance. A new feature called **company_age** was created using the year of establishment. Skewed numerical features were identified and transformed using the **Yeo-Johnson Power Transformation**. Feature scaling and encoding techniques were also applied to prepare the data for machine learning algorithms.

3.8 Data Transformation

The data transformation component converts validated data into a machine-learning-ready format. Encoding, scaling, skewness correction, and preprocessing pipelines were implemented using Scikit-learn's Pipeline and ColumnTransformer. The same transformation workflow is reused during training and prediction to ensure consistency.

3.9 Model Training and Hyperparameter Tuning

Machine learning models including K-Nearest Neighbors (KNN) and Random Forest were trained and evaluated. Hyperparameter tuning was performed using GridSearchCV with 3-fold Cross-Validation to identify the best parameter combinations. After tuning, KNN achieved the highest performance with an accuracy of approximately 94.53%.

3.10. Model Evaluation

The model evaluation component compares the newly trained model against the existing production model stored in AWS S3. Performance metrics such as F1-Score are used to determine whether the new model should replace the production model. Only models with improved performance are promoted for deployment.

3.11. Model Pusher

The Model Pusher component automates the deployment of trained model artifacts to AWS S3 cloud storage. It enables centralized model management, version control, and accessibility for future deployment and prediction services.

3.12. AWS Cloud Setup

AWS S3 was used as centralized cloud storage for trained models and preprocessing artifacts. This cloud-based architecture enables scalable deployment, easier model management, and seamless integration with the prediction service.

3.13. FastAPI Deployment

The application was deployed using FastAPI to provide a user-friendly web interface for visa approval prediction. FastAPI handles user requests, communicates with the prediction pipeline, and returns prediction results in real time. A retraining option was also integrated to trigger the training pipeline directly from the application.

3.14. Prediction Pipeline

The prediction pipeline processes new user input by converting it into a structured Pandas DataFrame and applying the same preprocessing and transformation steps used during training. The transformed data is then passed to the trained machine learning model to generate predictions.

3.15. Approval Prediction

The trained KNN model predicts whether a visa application is likely to be Certified or Denied based on the applicant and employer information provided by the user. The prediction result is displayed directly through the FastAPI web application.

4. Unit Test Cases

Test Case Description	Pre-Requisite	Expected Result
Verify whether the Application loads completely for the user when the URL is accessed	1. Application URL is accessible 2. Application is deployed	The Application should load completely for the user when the URL is accessed
Verify whether the User is able to sign up in the application	1. Application is accessible	The User should be able to sign up in the application
Verify whether user is able to successfully login to the application	1. Application is accessible 2. User is signed up to the application	User should be able to successfully login to the application
Verify whether user is able to see input fields on logging in	1. Application is accessible 2. User is signed up to the application 3. User is logged in to the application	User should be able to see input fields on logging in
Verify whether user is able to edit all input fields	1. Application is accessible 2. User is signed up to the application 3. User is logged in to the application	User should be able to edit all input fields
Verify whether user gets Submit button to submit the inputs	1. Application is accessible 2. User is signed up to the application 3. User is logged in to the application	User should get Submit button to submit the inputs
Verify whether user is presented with results on clicking submit	1. Application is accessible 2. User is signed up to the application 3. User is logged in to the application	User should be presented with results on clicking submit